

# Laporan Tugas Kecil 3

## IF2211 Strategi Algoritma

Ice Sliding Puzzle Solver



Nama / NIM : Wildan Abdurrahman Ghazali / 13524054

## Daftar Isi

|                                                       |    |
|-------------------------------------------------------|----|
| Daftar Isi.....                                       | 1  |
| A. Dasar Teori.....                                   | 2  |
| 1. Ice Sliding Puzzle.....                            | 2  |
| 2. Uniform Cost Search.....                           | 2  |
| 3. Greedy Best First Search.....                      | 3  |
| 4. A*.....                                            | 4  |
| B. Analisis Algoritma.....                            | 5  |
| 1. Definisi State, $f(n)$ , $g(n)$ , dan $h(n)$ ..... | 5  |
| 2. Admissibility Heuristic A*.....                    | 5  |
| 3. Perbandingan Teoretis.....                         | 5  |
| C. Implementasi.....                                  | 6  |
| D. Hasil Pengujian.....                               | 31 |
| 1. Kasus Uji.....                                     | 31 |
| 2. Analisis Hasil Pengujian.....                      | 42 |
| Lampiran.....                                         | 45 |

## A. Dasar Teori

### 1. Ice Sliding Puzzle

Ice Sliding Puzzle adalah permainan *puzzle* pencarian lintasan pada papan grid. Pemain atau aktor bergerak dari posisi awal Z menuju titik tujuan O. Berbeda dengan pathfinding grid biasa, satu aksi tidak berhenti pada petak berikutnya, tetapi membuat aktor terus meluncur secara horizontal atau vertikal sampai tepat sebelum menabrak rintangan X.

Input program terdiri atas ukuran papan N M, N baris dan M kolom konfigurasi papan, diikuti N baris dengan M kolom matriks cost. Simbol \* adalah tile yang dapat dilewati, X adalah rintangan, L adalah lava, Z adalah posisi awal, O adalah goal, dan digit 0 sampai 9 adalah checkpoint wajib. Digit harus dilewati sesuai urutan. Jika aktor melewati digit yang belum waktunya, melewati lava, atau bergerak ke arah yang tidak memiliki rintangan penghenti sebelum batas papan, gerakan dianggap invalid.

### 2. Uniform Cost Search

Uniform Cost Search atau UCS adalah algoritma pencarian berbasis priority queue yang selalu memilih simpul dengan nilai biaya kumulatif terkecil dari simpul awal. Pada algoritma ini, prioritas setiap simpul ditentukan oleh nilai  $g(n)$ , yaitu total biaya aktual yang telah dikeluarkan dari kondisi awal sampai ke simpul n. Secara umum, fungsi evaluasi UCS dapat dituliskan sebagai berikut.

$$f(n) = g(n)$$

dengan  $g(n)$  menyatakan biaya kumulatif dari simpul awal menuju simpul n.

Pada implementasi, node merepresentasikan state (baris, kolom, angka\_berikutnya), sedangkan successor diperoleh dengan mencoba empat arah sliding, yaitu atas, kanan, bawah, dan kiri.

Langkah kerja UCS pada implementasi adalah sebagai berikut:

- Program memulai dari posisi Z,
- Program memasukkan state awal ke pqueue (priority queue) dengan prioritas  $g(n)=0$ ,
- Program lalu berulang kali mengambil state dengan  $g(n)$  terkecil.

- d. Untuk setiap state yang diekspansi, program melakukan simulasi empat arah sliding:
  - i. Program menghitung cost seluruh tile yang dilewati,
  - ii. Program memperbarui angka berikutnya apabila checkpoint yang benar dilewati,
  - iii. Program memasukkan successor hanya apabila cost kumulatifnya lebih baik daripada catatan sebelumnya.
- e. Pencarian berhenti ketika state yang dikeluarkan dari priority queue berada di O dan semua angka wajib sudah dilewati.

Karena semua cost input divalidasi tidak negatif, UCS bersifat optimal terhadap total cost solusi. Kekurangannya, UCS tidak memiliki informasi arah menuju goal sehingga dapat meninjau banyak state pada papan yang besar atau bercabang.

### 3. Greedy Best First Search

Greedy Best First Search atau GBFS adalah algoritma pencarian yang memilih simpul berdasarkan nilai heuristik terkecil. Berbeda dengan UCS yang mempertimbangkan biaya aktual dari awal, GBFS hanya memperhatikan estimasi jarak atau biaya dari simpul saat ini menuju target. Fungsi evaluasi GBFS dapat dituliskan sebagai berikut.

$$f(n) = h(n)$$

dengan  $h(n)$  menyatakan nilai heuristik dari simpul  $n$  menuju tujuan.

Langkah kerja GBFS adalah:

- a. Program menghitung  $h(n)$  pada state awal,
- b. Program mengambil node dengan  $h(n)$  terkecil dari pqueue.
- c. Setelah successor valid dibangkitkan melalui simulasi sliding, setiap successor diberi nilai heuristic baru dan dimasukkan ke priority queue.

Kelemahan utama GBFS adalah dapat memilih jalur yang terlihat dekat dengan tujuan, tetapi sebenarnya memiliki biaya traversal yang besar. Akibatnya, GBFS dapat menemukan solusi dengan cepat apabila heuristik yang digunakan cukup baik, tetapi algoritma ini tidak menjamin solusi dengan total cost

minimum. Oleh karena itu, GBFS lebih cocok digunakan ketika kecepatan pencarian lebih diprioritaskan daripada optimalitas cost solusi.

#### 4. A\*

A\* adalah algoritma pencarian yang menggabungkan prinsip UCS dan GBFS. Algoritma ini mempertimbangkan dua komponen utama, yaitu biaya aktual dari start ke simpul saat ini dan estimasi biaya dari simpul saat ini menuju tujuan. Fungsi evaluasi A\* dapat dituliskan sebagai berikut.

$$f(n) = g(n) + h(n)$$

dengan  $g(n)$  menyatakan total cost aktual dari simpul awal ke simpul  $n$ , sedangkan  $h(n)$  menyatakan estimasi cost dari simpul  $n$  menuju target.

Pada implementasi, A\* menggunakan fungsi prioritas yang berbeda dari UCS dan GBFS. Setiap kali suatu successor dibangkitkan, program menghitung cost aktual dari seluruh tile yang dilewati selama gerakan sliding. Cost tersebut ditambahkan ke nilai  $g(n)$  sebelumnya. Setelah itu, nilai heuristik  $h(n)$  dihitung berdasarkan estimasi jarak menuju target berikutnya, sehingga diperoleh nilai prioritas  $f(n)$ .

Implementasi menyediakan tiga heuristic. H1 menghitung jarak Manhattan dari posisi aktor ke target berikutnya, yaitu digit berikutnya atau goal. H2 menghitung estimasi berantai dari posisi saat ini menuju semua digit tersisa secara urut lalu ke goal. H3 mengalikan H2 dengan  $\min\_passable\_cost$ , yaitu cost minimum dari tile yang dapat dilalui.

Dengan pendekatan tersebut, A\* mampu menyeimbangkan dua aspek pencarian. Dari sisi UCS, A\* tetap memperhatikan total cost aktual agar tidak sembarangan memilih jalur yang mahal. Dari sisi GBFS, A\* menggunakan heuristik agar proses pencarian lebih terarah menuju tujuan. Oleh karena itu, A\* umumnya lebih efisien daripada UCS dan lebih aman daripada GBFS dalam hal kualitas solusi.

A\* dapat menjamin solusi optimal apabila heuristik yang digunakan bersifat admissible, yaitu tidak pernah melebihi-lebihkan estimasi cost sebenarnya menuju tujuan.

## B. Analisis Algoritma

### 1. Definisi State, $f(n)$ , $g(n)$ , dan $h(n)$

- State didefinisikan sebagai tuple (row, col, next\_number). Komponen row dan col menyatakan posisi aktor, sedangkan next\_number menyatakan digit berikutnya yang wajib dilewati.
- $g(n)$ : total cost traversal aktual dari state awal sampai state n. Cost ini berasal dari penjumlahan cost semua tile yang dilewati pada setiap gerakan sliding.
- $h(n)$ : estimasi cost tersisa dari state n menuju target berikutnya atau menuju goal setelah seluruh digit dilewati.
- UCS menggunakan prioritas  $f(n)=g(n)$ , sehingga node dengan cost aktual terkecil diproses lebih dulu.
- GBFS menggunakan prioritas  $f(n)=h(n)$ , sehingga pencarian hanya berdasarkan estimasi heuristic.
- A\* menggunakan prioritas  $f(n)=g(n)+h(n)$ , sehingga cost aktual dan estimasi sisa jarak dipertimbangkan sekaligus.

### 2. Admissibility Heuristic A\*

Heuristic H1 dan H2 berbasis jarak Manhattan. Keduanya mengabaikan rintangan, lava, dan syarat berhenti sebelum X, sehingga secara struktur papan biasanya menghasilkan estimasi yang tidak lebih panjang daripada lintasan nyata. Karena program memperbolehkan cost 0, H1 dan H2 tidak selalu admissible untuk semua input valid: estimasi jarak dapat lebih besar daripada cost aktual apabila lintasan tersisa melewati tile berbiaya 0.

H3 didefinisikan sebagai H2 dikali min\_passable\_cost. Karena seluruh cost divalidasi tidak negatif, setiap tile valid yang dilewati memiliki cost minimal sebesar min\_passable\_cost. Maka H3 tidak melebihi-lebihkan cost optimal tersisa. Jika terdapat tile passable dengan cost 0, nilai H3 dapat menjadi 0 sehingga heuristic menjadi lemah, tetapi tetap admissible.

### 3. Perbandingan Teoretis

Berikut adalah tabel perbandingan setiap algoritma *pathfinding* berdasarkan kelebihan, kekurangan, dan kesesuaian pada puzzle:

| Algoritma | Kelebihan                                                  | Kekurangan                                                  | Kesesuaian pada puzzle                                         |
|-----------|------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|
| UCS       | Optimal untuk cost non-negatif, cocok pada papan berbobot. | Dapat meninjau banyak state karena tidak memakai heuristic. | Baik sebagai baseline optimalitas dan pembandingan cost.       |
| GBFS      | Cepat jika heuristic tepat, pqueue sering lebih kecil.     | Tidak optimal karena mengabaikan $g(n)$ .                   | Cocok untuk solusi cepat, tetapi berisiko memilih jalur mahal. |
| A*        | Lebih seimbang, optimal jika heuristic admissible.         | Kualitas sangat bergantung pada heuristic.                  | Cocok untuk pencarian cost-aware yang tetap terarah.           |

Tabel 1. Perbandingan algoritma *pathfinding*

### C. Implementasi

Berikut adalah *source code* program dalam bahasa python (dapat dilihat juga pada repository Github yang tertera pada lampiran) :

```
main.py
```

```

import sys
from pathlib import Path

if __package__ in (None, ""):
    sys.path.insert(0,
str(Path(__file__).resolve().parents[1]))
    __package__ = "src"

from .models import BoardFrame, InputValidationError,
SearchResult
from .output import save_solution
from .parser import parse_file
from .search import solve

HEURISTIC_CHOICES = ("H1", "H2", "H3")

def format_number(value: float) -> str:
    if abs(value - round(value)) < 1e-9:
        return str(int(round(value)))
    return f"{value:.3f}"

def read_line(prompt: str) -> str:
    print(prompt)
    return input().strip().strip('\"')

def normalize_algorithm(raw: str) -> str | None:
    normalized = raw.strip().upper()
    if normalized in {"UCS", "GBFS", "A*"}:
        return normalized
    return None

def prompt_algorithm() -> str:
    while True:
        raw = read_line(">> Algoritma apa yang anda
pilih? (UCS/GBFS/A*)")
        algorithm = normalize_algorithm(raw)
        if algorithm is not None:
            return algorithm
        print("Pilihan algoritma tidak valid. Pilihan:
UCS, GBFS, A*.")

```

```

def prompt_heuristic() -> str:
    while True:
        raw = read_line(">> Heuristic apa yang anda
pilih? (H1/H2/H3) ")
        heuristic = raw.strip().upper()
        if heuristic in HEURISTIC_CHOICES:
            return heuristic
        print("Pilihan heuristic tidak valid. Pilihan:
H1, H2, H3.")

def prompt_yes_no(prompt: str) -> bool:
    while True:
        answer = read_line(prompt).lower()
        if answer == "ya":
            return True
        if answer in "tidak":
            return False
        print("Jawaban tidak valid. Masukkan Ya atau
Tidak.")

def print_frame(frame: BoardFrame) -> None:
    print(frame.title)
    print(frame.board_text)

def print_result(result: SearchResult) -> None:
    print()
    if result.found:
        print(f"Solusi Yang Ditemukan :
{result.solution_string}")
        print(f"Cost dari Solusi : {result.total_cost}")
        print()
        for frame in result.solution_frames:
            print_frame(frame)
        print()
    else:
        print("Solusi tidak ditemukan.")
        if result.error_message:
            print(result.error_message)
        print()

    print(f">> Waktu eksekusi:
{format_number(result.execution_time_ms)} ms")

```

```

        print(f">> Banyak iterasi yang dilakukan:
{result.iterations} iterasi")
        print()

def read_step(max_step: int, prompt: str) -> int:
    while True:
        raw = read_line(prompt)
        try:
            step = int(raw)
        except ValueError:
            print("Step harus berupa integer.")
            continue
        if 0 <= step <= max_step:
            return step
        print(f"Step harus berada pada rentang 0 sampai
{max_step}.")

def show_playback_step(frames: list[BoardFrame],
current_step: int) -> None:
    max_step = len(frames) - 1
    print()
    print(f"Playback Step {current_step} / {max_step}")
    print_frame(frames[current_step])
    print()

def run_playback(frames: list[BoardFrame]) -> None:
    if not frames:
        print("Playback solusi tidak tersedia karena
solusi tidak ditemukan.")
        return

    max_step = len(frames) - 1
    current_step = read_step(max_step, ">> Pada step
berapa anda ingin melakukan playback :")
    show_playback_step(frames, current_step)

    while True:
        print("Playback command:")
        print("[n] next, [p] previous, [j] jump, [q]
quit")
        command = read_line(">>").lower()

        if command == "q":

```

```

        print("Playback selesai.")
        return
    if command == "n":
        if current_step < max_step:
            current_step += 1
        else:
            print("Sudah berada di step terakhir.")
            show_playback_step(frames, current_step)
            continue
    if command == "p":
        if current_step > 0:
            current_step -= 1
        else:
            print("Sudah berada di step awal.")
            show_playback_step(frames, current_step)
            continue
    if command == "j":
        current_step = read_step(max_step, ">>
Masukan step tujuan :")
        show_playback_step(frames, current_step)
        continue

    print("Command tidak valid. Gunakan n, p, j,
atau q.")

def normalize_output_path(raw_path: str, default_path:
Path) -> Path:
    path = default_path if not raw_path else
Path(raw_path.strip().strip(''))
    if path.suffix == "":
        path = path.with_suffix(".txt")
    if path.suffix.lower() != ".txt":
        raise ValueError("File output harus berekstensi
.txt.")
    return path

def save_result(result: SearchResult, input_path: str)
-> bool:
    if not prompt_yes_no(">> Apakah Anda ingin menyimpan
solusi? (Ya/Tidak) :"):
        return True

    default_path = Path("test") / "solution.txt"
    while True:

```

```

        raw_path = read_line(
            ">> Masukkan path file output (.txt),
            kosongkan untuk default test/solution.txt :")
        )
        try:
            output_path =
normalize_output_path(raw_path, default_path)
            save_solution(result, output_path,
input_path=input_path)
        except (OSError, ValueError) as exc:
            print(f"Gagal menyimpan solusi: {exc}")
            retry = prompt_yes_no(">> Coba path lain?
(Ya/Tidak) :")
            if not retry:
                return False
            continue

        print(f">> Solusi disimpan pada
{output_path.resolve()}")
        return True

def run_cli() -> int:
    try:
        file_path = read_line(">> Masukkan file input :")
        board = parse_file(file_path)

        algorithm = prompt_algorithm()
        heuristic = None
        if algorithm in {"GBFS", "A*"}:
            heuristic = prompt_heuristic()

        result = solve(
            board,
            algorithm=algorithm,
            heuristic_name=heuristic,
            collect_iteration_frames=False,
        )

        print_result(result)

        if prompt_yes_no(">> Apakah Anda ingin melakukan
        playback? (Ya/Tidak) :"):
            run_playback(result.solution_frames)

```

```
        if not save_result(result, board.source_path or
file_path):
            return 1
        return 0

    except (InputValidationError, ValueError) as exc:
        print(f"Error: {exc}")
        return 1
    except KeyboardInterrupt:
        print("\nProgram dihentikan")
        return 1

if __name__ == "__main__":
    raise SystemExit(run_cli())
```

models.py

```

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Optional

Position = tuple[int, int]
State = tuple[int, int, int]


class InputValidationError(Exception):
    """Raised when an input file cannot be used as a
    puzzle board."""

@dataclass(frozen=True)
class Board:
    rows: int
    cols: int
    base_grid: tuple[str, ...]
    costs: tuple[tuple[int, ...], ...]
    start_pos: Position
    goal_pos: Position
    digit_positions: dict[int, tuple[Position, ...]] =
field(default_factory=dict)
    source_path: Optional[str] = None
    min_passable_cost: int = 0

    @property
    def required_count(self) -> int:
        return max(self.digit_positions.keys(),
default=-1) + 1

    @property
    def max_digit(self) -> int:
        return self.required_count - 1

@dataclass(frozen=True)
class MoveResult:
    new_state: State
    move_cost: int
    traversed_cells: list[Position]
    action: str

@dataclass

```

```

class Node:
    state: State
    g: float
    h: float = 0.0
    action: Optional[str] = None
    parent: Optional["Node"] = None
    move_cells: list[Position] =
field(default_factory=list)

    @property
    def f(self) -> float:
        return self.g + self.h

@dataclass(frozen=True)
class BoardFrame:
    board_text: str
    actor_pos: Position
    state: State
    title: str
    action: Optional[str]
    step_index: int
    g: float
    h: float
    f: float
    next_number: int
    kind: str

@dataclass
class SearchResult:
    found: bool
    algorithm: str
    heuristic_name: Optional[str]
    solution_actions: list[str] =
field(default_factory=list)
    solution_string: str = ""
    total_cost: int = 0
    iterations: int = 0
    execution_time_ms: float = 0.0
    solution_nodes: list[Node] =
field(default_factory=list)
    solution_frames: list[BoardFrame] =
field(default_factory=list)
    iteration_frames: list[BoardFrame] =
field(default_factory=list)

```

```
expanded_states_count: int = 0  
error_message: Optional[str] = None
```

```
board.py
```

```

from __future__ import annotations

from .models import Board, BoardFrame, MoveResult, Node,
Position, State

DIRECTIONS: dict[str, tuple[int, int]] = {
    "U": (-1, 0),
    "R": (0, 1),
    "D": (1, 0),
    "L": (0, -1),
}

DIRECTION_ORDER: tuple[str, ...] = ("U", "R", "D", "L")

def start_state(board: Board) -> State:
    return (board.start_pos[0], board.start_pos[1], 0)

def in_bounds(board: Board, row: int, col: int) -> bool:
    return 0 <= row < board.rows and 0 <= col <
board.cols

def tile_at(board: Board, row: int, col: int) -> str:
    return board.base_grid[row][col]

def is_goal(board: Board, state: State) -> bool:
    row, col, next_number = state
    return (row, col) == board.goal_pos and next_number
== board.required_count

def simulate_move(board: Board, state: State, direction:
str) -> MoveResult | None:
    if direction not in DIRECTIONS:
        return None

    row, col, next_number = state
    dr, dc = DIRECTIONS[direction]
    total_cost = 0
    moved = False
    traversed_cells: list[Position] = []

    while True:

```

```

        next_row = row + dr
        next_col = col + dc

        if not in_bounds(board, next_row, next_col):
            return None

        tile = tile_at(board, next_row, next_col)
        if tile == "X":
            if not moved:
                return None
            return MoveResult(
                new_state=(row, col, next_number),
                move_cost=total_cost,
                traversed_cells=traversed_cells,
                action=direction,
            )

        moved = True
        traversed_cells.append((next_row, next_col))
        total_cost += board.costs[next_row][next_col]

        if tile == "L":
            return None

        if tile.isdigit():
            digit = int(tile)
            if digit < next_number:
                pass
            elif digit == next_number:
                next_number += 1
            else:
                return None

        row = next_row
        col = next_col

def render_board_lines(
    board: Board,
    actor_pos: Position | None = None,
    next_number: int | None = None,
) -> list[str]:
    lines: list[str] = []
    for row_index, row_text in
enumerate(board.base_grid):
        chars = list(row_text)

```

```

        if next_number is not None:
            for col_index, char in enumerate(chars):
                if char.isdigit() and int(char) <
next_number:
                    chars[col_index] = "*"
            if actor_pos is not None and row_index ==
actor_pos[0]:
                chars[actor_pos[1]] = "Z"
            lines.append("".join(chars))
        return lines

def render_board_text(
    board: Board,
    actor_pos: Position | None = None,
    next_number: int | None = None,
) -> str:
    return "\n".join(render_board_lines(board,
actor_pos, next_number))

def make_frame(
    board: Board,
    node: Node,
    title: str,
    step_index: int,
    kind: str,
    action: str | None = None,
) -> BoardFrame:
    row, col, next_number = node.state
    return BoardFrame(
        board_text=render_board_text(board, (row, col),
next_number),
        actor_pos=(row, col),
        state=node.state,
        title=title,
        action=action,
        step_index=step_index,
        g=node.g,
        h=node.h,
        f=node.f,
        next_number=next_number,
        kind=kind,
    )

```

```
search.py
```

```

from __future__ import annotations

import heapq
import itertools
import time

from .board import DIRECTION_ORDER, is_goal, make_frame,
simulate_move, start_state
from .heuristic import get_heuristic
from .models import Board, BoardFrame, Node,
SearchResult

SUPPORTED_ALGORITHMS = ("UCS", "GBFS", "A*")
EPSILON = 1e-9

def priority(algorithm: str, node: Node) -> float:
    if algorithm == "UCS":
        return node.g
    if algorithm == "GBFS":
        return node.h
    return node.f

def reconstruct_nodes(goal_node: Node) -> list[Node]:
    nodes: list[Node] = []
    current: Node | None = goal_node
    while current is not None:
        nodes.append(current)
        current = current.parent
    nodes.reverse()
    return nodes

def build_solution_frames(board: Board, nodes:
list[Node]) -> list[BoardFrame]:
    frames: list[BoardFrame] = []
    for index, node in enumerate(nodes):
        if index == 0:
            title = "Initial"
            action = None
        else:
            action = node.action
            title = f"Step {index} : {action}"
        frames.append(

```

```

        make_frame(
            board=board,
            node=node,
            title=title,
            step_index=index,
            kind="solution",
            action=action,
        )
    )
    return frames

def solve(
    board: Board,
    algorithm: str = "UCS",
    heuristic_name: str | None = "H1",
    collect_iteration_frames: bool = True,
) -> SearchResult:

    active_heuristic_name = None if algorithm == "UCS"
    else (heuristic_name or "H1")
    heuristic = None if algorithm == "UCS" else
    get_heuristic(active_heuristic_name)

    start = start_state(board)
    start_h = 0.0 if heuristic is None else
    float(heuristic(board, start))
    root = Node(state=start, g=0.0, h=start_h)

    counter = itertools.count()
    frontier: list[tuple[float, int, Node]] = []
    heapq.heappush(frontier, (priority(algorithm, root),
    next(counter), root))
    best_cost: dict[tuple[int, int, int], float] =
    {start: 0.0}
    iterations = 0
    iteration_frames: list[BoardFrame] = []
    goal_node: Node | None = None

    start_time = time.perf_counter()

    while frontier:
        _, _, node = heapq.heappop(frontier)
        if node.g > best_cost.get(node.state,
        float("inf")) + EPSILON:
            continue

```

```

        iterations += 1
        if collect_iteration_frames:
            iteration_frames.append(
                make_frame(
                    board=board,
                    node=node,
                    title=f"Iteration {iterations}",
                    step_index=iterations,
                    kind="iteration",
                    action=node.action,
                )
            )

        if is_goal(board, node.state):
            goal_node = node
            break

        for direction in DIRECTION_ORDER:
            move = simulate_move(board, node.state,
direction)
            if move is None:
                continue

            new_g = node.g + move.move_cost
            if new_g + EPSILON >=
best_cost.get(move.new_state, float("inf")):
                continue

            new_h = 0.0 if heuristic is None else
float(heuristic(board, move.new_state))
            child = Node(
                state=move.new_state,
                g=new_g,
                h=new_h,
                action=direction,
                parent=node,
                move_cells=move.traversed_cells,
            )
            best_cost[move.new_state] = new_g
            heapq.heappush(frontier,
(priority(algorithm, child), next(counter), child))

        execution_time_ms = (time.perf_counter() -
start_time) * 1000

```

```

    if goal_node is None:
        return SearchResult(
            found=False,
            algorithm=algorithm,
            heuristic_name=active_heuristic_name,
            iterations=iterations,
            execution_time_ms=execution_time_ms,
            iteration_frames=iteration_frames,
            expanded_states_count=iterations,
            error_message="Tidak ditemukan solusi.",
        )

    solution_nodes = reconstruct_nodes(goal_node)
    solution_actions = [node.action for node in
        solution_nodes[1:] if node.action is not None]
    return SearchResult(
        found=True,
        algorithm=algorithm,
        heuristic_name=active_heuristic_name,
        solution_actions=solution_actions,
        solution_string="".join(solution_actions),
        total_cost=int(goal_node.g),
        iterations=iterations,
        execution_time_ms=execution_time_ms,
        solution_nodes=solution_nodes,
        solution_frames=build_solution_frames(board,
        solution_nodes),
        iteration_frames=iteration_frames,
        expanded_states_count=iterations,
    )

```

```

heuristic.py

```

```

from __future__ import annotations

from collections.abc import Callable

from .models import Board, Position, State

HeuristicFunction = Callable[[Board, State], float]

def manhattan(first: Position, second: Position) -> int:
    return abs(first[0] - second[0]) + abs(first[1] -
second[1])

def next_targets(board: Board, state: State) ->
tuple[Position, ...]:
    row, col, next_number = state
    if next_number < board.required_count:
        return board.digit_positions.get(next_number,
())
    return (board.goal_pos,)

def h1(board: Board, state: State) -> float:
    row, col, _ = state
    current = (row, col)
    targets = next_targets(board, state)
    if not targets:
        return 0.0
    return float(min(manhattan(current, target) for
target in targets))

def h2(board: Board, state: State) -> float:
    row, col, next_number = state
    distances: dict[Position, int] = {(row, col): 0}

    for digit in range(next_number,
board.required_count):
        targets = board.digit_positions.get(digit, ())
        if not targets:
            return 0.0
        next_distances: dict[Position, int] = {}
        for target in targets:
            next_distances[target] = min(
                distance + manhattan(position, target)

```

```

        for position, distance in
distances.items()
    )
    distances = next_distances

    return float(
        min(
            distance + manhattan(position,
board.goal_pos)
            for position, distance in distances.items()
        )
    )

def h3(board: Board, state: State) -> float:
    return h2(board, state) * board.min_passable_cost

HEURISTICS: dict[str, HeuristicFunction] = {
    "H1": h1,
    "H2": h2,
    "H3": h3,
}

def get_heuristic(name: str | None) ->
HeuristicFunction:
    if not name:
        return h1
    key = name.strip().upper()
    if key not in HEURISTICS:
        available = ", ".join(sorted(HEURISTICS))
        raise ValueError(f"Heuristic tidak dikenal:
{name}. Pilihan: {available}.")
    return HEURISTICS[key]

```

```

parser.py

```

```

from __future__ import annotations

from pathlib import Path

from .models import Board, InputValidationError,
Position

VALID_TILES = set("X*LZO0123456789")

def parse_positive_int(value: str, label: str) -> int:
    try:
        parsed = int(value)
    except ValueError as exc:
        raise InputValidationError(f"{label} harus
berupa integer.") from exc
    if parsed <= 0:
        raise InputValidationError(f"{label} harus
bernilai positif.")
    return parsed

def parse_cost_row(line: str, expected_cols: int,
row_number: int) -> tuple[int, ...]:
    parts = line.split()
    if len(parts) != expected_cols:
        raise InputValidationError(
            f"Baris cost ke-{row_number} harus berisi
tepat {expected_cols} integer."
        )

    costs: list[int] = []
    for col_number, part in enumerate(parts, start=1):
        try:
            value = int(part)
        except ValueError as exc:
            raise InputValidationError(
                f"Cost pada baris {row_number}, kolom
{col_number} harus integer."
            ) from exc
        if value < 0:
            raise InputValidationError(
                f"Cost pada baris {row_number}, kolom
{col_number} tidak boleh negatif."
            )
        costs.append(value)

```

```

        return tuple(costs)

def parse_file(file_path: str | Path) -> Board:
    path = Path(file_path)
    if path.suffix.lower() != ".txt":
        raise InputValidationError("File input harus
berekstensi .txt.")
    if not path.exists():
        raise InputValidationError(f"File tidak
ditemukan: {path}")
    if not path.is_file():
        raise InputValidationError(f"Path bukan file:
{path}")

    try:
        raw_lines =
path.read_text(encoding="utf-8").splitlines()
    except OSError as exc:
        raise InputValidationError(f"Gagal membaca file:
{exc}") from exc

    while raw_lines and raw_lines[-1].strip() == "":
        raw_lines.pop()

    if not raw_lines:
        raise InputValidationError("File kosong.")

    header = raw_lines[0].strip().split()
    if len(header) != 2:
        raise InputValidationError("Baris pertama harus
berisi tepat dua integer: N M.")

    rows = parse_positive_int(header[0], "N")
    cols = parse_positive_int(header[1], "M")
    expected_line_count = 1 + rows + rows

    if len(raw_lines) < expected_line_count:
        raise InputValidationError(
            f"Jumlah baris tidak lengkap. Dibutuhkan
{rows} baris board dan {rows} baris cost."
        )
    if len(raw_lines) > expected_line_count:
        raise InputValidationError("File memiliki baris
tambahan setelah matrix cost.")

```

```

    board_lines = [line.strip() for line in raw_lines[1
: 1 + rows]]
    cost_lines = [line.strip() for line in raw_lines[1 +
rows : expected_line_count]]

    start_positions: list[Position] = []
    goal_positions: list[Position] = []
    digit_positions: dict[int, list[Position]] = {}
    base_grid: list[str] = []

    for row_index, line in enumerate(board_lines):
        display_row = row_index + 1
        if len(line) != cols:
            raise InputValidationError(
                f"Baris board ke-{display_row} harus
memiliki panjang tepat {cols} karakter."
            )

        chars = list(line)
        for col_index, char in enumerate(chars):
            if char not in VALID_TILES:
                raise InputValidationError(
                    f"Karakter '{char}' pada baris
{display_row}, kolom {col_index + 1} tidak valid."
                )
            if char == "Z":
                start_positions.append((row_index,
col_index))
            elif char == "*":
                chars[col_index] = "*"
            elif char == "O":
                goal_positions.append((row_index,
col_index))
            elif char.isdigit():
                digit_positions.setdefault(int(char),
[]).append((row_index, col_index))
            base_grid.append("".join(chars))

    if len(start_positions) != 1:
        raise InputValidationError("Board harus memiliki
tepat satu posisi awal Z.")
    if len(goal_positions) != 1:
        raise InputValidationError("Board harus memiliki
tepat satu titik tujuan O.")

    if digit_positions:
        max_digit = max(digit_positions)

```

```

        missing = [str(digit) for digit in
range(max_digit + 1) if digit not in digit_positions]
        if missing:
            raise InputValidationError(
                "Angka pada board harus lengkap dari 0
sampai max digit. "
                f"Angka yang hilang: {'',
'.'.join(missing)}."
            )

        costs = tuple(parse_cost_row(line, cols, idx) for
idx, line in enumerate(cost_lines, start=1))

        passable_costs: list[int] = []
        for row_index, row_text in enumerate(base_grid):
            for col_index, char in enumerate(row_text):
                if char not in {"X", "L"}:

passable_costs.append(costs[row_index][col_index])

        min_passable_cost = min(passable_costs) if
passable_costs else 0
        frozen_digit_positions = {
            digit: tuple(positions) for digit, positions in
digit_positions.items()
        }

        return Board(
            rows=rows,
            cols=cols,
            base_grid=tuple(base_grid),
            costs=costs,
            start_pos=start_positions[0],
            goal_pos=goal_positions[0],
            digit_positions=frozen_digit_positions,
            source_path=str(path),
            min_passable_cost=min_passable_cost,
        )

```

output.py

```

from __future__ import annotations

from pathlib import Path

from .models import SearchResult

def format_number(value: float) -> str:
    if abs(value - round(value)) < 1e-9:
        return str(int(round(value)))
    return f"{value:.3f}"

def header_lines(result: SearchResult, input_path: str |
None = None) -> list[str]:
    lines: list[str] = []
    if input_path:
        lines.append(f"Input File: {input_path}")
    lines.append(f"Algoritma: {result.algorithm}")
    if result.heuristic_name:
        lines.append(f"Heuristic:
{result.heuristic_name}")
    else:
        lines.append("Heuristic: -")
    lines.append("")
    return lines

def format_solution(result: SearchResult, input_path:
str | None = None) -> str:
    lines = header_lines(result, input_path)

    if not result.found:
        lines.extend(
            [
                "Status: Tidak ditemukan solusi.",
                f"Waktu Eksekusi:
{format_number(result.execution_time_ms)} ms",
                f"Banyak Iterasi: {result.iterations}",
                "",
            ]
        )
    return "\n".join(lines)

    lines.extend(
        [

```

```

        "Status: Solusi ditemukan.",
        f"Solusi Yang Ditemukan:
{result.solution_string}",
        f"Cost dari Solusi: {result.total_cost}",
        f"Waktu Eksekusi:
{format_number(result.execution_time_ms)} ms",
        f"Banyak Iterasi: {result.iterations}",
        "",
    ]
)

for frame in result.solution_frames:
    lines.append(frame.title)
    lines.append(frame.board_text)
    lines.append("")

return "\n".join(lines).rstrip() + "\n"

def save_solution(
    result: SearchResult,
    file_path: str | Path,
    input_path: str | None = None,
) -> None:
    path = Path(file_path)
    if path.parent != Path("."):
        path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(format_solution(result, input_path),
encoding="utf-8")

```

## D. Hasil Pengujian

### 1. Kasus Uji

Berikut adalah tabel hasil pengujian yang dilakukan:

| No | Hasil                  |
|----|------------------------|
| 1. | Menguji fitur playback |

File input:

```
7 7
XXXXXX
X0***X
X**X*1X
X****OX
X****LX
XZ**X*X
XXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

Output:

```
>> Masukan file input :  
test/testcase1.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)  
UCS
```

```
Solusi Yang Ditemukan : UDRURULDRUR  
Cost dari Solusi : 146
```

```
Initial
```

```
XXXXXXX
```

```
X0***X
```

```
X**X*1X
```

```
X***0X
```

```
X***LX
```

```
XZ**X*X
```

```
XXXXXXX
```

```
Step 1 : U
```

```
XXXXXXX
```

```
XZ***X
```

```
X**X*1X
```

```
X***0X
```

```
X***LX
```

```
X***X*X
```

```
XXXXXXX
```

```
Step 2 : D
```

```
XXXXXXX
```

```
X***X
```

```
X**X*1X
```

```
X***0X
```

```
X***LX
```

```
XZ**X*X
```

```
XXXXXXX
```

```

>> Waktu eksekusi: 0.262 ms
>> Banyak iterasi yang dilakukan: 18 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
ya
>> Pada step berapa anda ingin melakukan playback :
2

Playback Step 2 / 11
Step 2 : D
XXXXXXX
X*****X
X**X*1X
X****0X
X****LX
XZ**X*X
XXXXXXX

Playback command:
[n] next, [p] previous, [j] jump, [q] quit
>>
n

Playback Step 3 / 11
Step 3 : R
XXXXXXX
X*****X
X**X*1X
X****0X
X****LX
X**ZX*X
XXXXXXX

Playback command:
[n] next, [p] previous, [j] jump, [q] quit
>>
p

Playback Step 2 / 11
Step 2 : D
XXXXXXX
X*****X
X**X*1X
X****0X
X****LX
XZ**X*X
XXXXXXX

```

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <pre> Playback command: [n] next, [p] previous, [j] jump, [q] quit &gt;&gt; q Playback selesai. &gt;&gt; Apakah Anda ingin menyimpan solusi? (Ya/Tidak) : Ya &gt;&gt; Masukkan path file output (.txt), kosongkan untuk default result/solution.txt : test/solution1.txt &gt;&gt; Solusi disimpan pada C:\Users\wilda\GitHub\Tucil3_13524054\test\solution1.txt PS C:\Users\wilda\GitHub\Tucil3_13524054&gt; </pre>                                                                        |
| 2. | <p>Menguji perbandingan UCS, GBFS, dan A*</p> <p>File input:</p> <pre> testcase2.txt 7 11 XXXXXXXXXXXX XXXXXXXXXXXX X*****XX X*XX*XXX*XX X*XXZ***OXX X****XXXXXX XXXXXXXXXXXX 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 0 0 0 0 0 0 0 0 999 999 999 0 999 999 2 999 999 999 0 999 999 999 0 999 999 0 30 30 30 0 999 999 999 0 0 0 0 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 </pre> <p>Output:<br/>(UCS)</p> |

```
test/testcase2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS
```

```
Solusi Yang Ditemukan : DLURD
Cost dari Solusi : 0
```

```
Initial
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XXZ***OXX
X****XXXXX
XXXXXXXXXXXX
```

```
Step 1 : D
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XX****OXX
X***ZXXXXX
XXXXXXXXXXXX
```

```
Step 2 : L
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XX****OXX
XZ***XXXXX
XXXXXXXXXXXX
```

```
Step 3 : U
XXXXXXXXXXXX
XXXXXXXXXXXX
XZ*****XX
X*XX*XX*XX
X*XX****OXX
X****XXXXX
XXXXXXXXXXXX
```

```
Step 4 : R
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****ZXX
X*XX*XX*XX
X*XX****OXX
X****XXXXX
XXXXXXXXXXXX
```

(GBFS)

```
test/testcase2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : R
Cost dari Solusi : 90

Initial
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XXZ***OXX
X***XXXXXX
XXXXXXXXXXXX

Step 1 : R
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XX***ZXX
X***XXXXXX
XXXXXXXXXXXX

>> Waktu eksekusi: 0.076 ms
>> Banyak iterasi yang dilakukan: 2 iterasi
```

(A\*)

```

A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : URD
Cost dari Solusi : 2

Initial
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XXZ***OXX
X****XXXXXX
XXXXXXXXXXXX

Step 1 : U
XXXXXXXXXXXX
XXXXXXXXXXXX
X***Z****XX
X*XX*XX*XX
X*XX****OXX
X****XXXXXX
XXXXXXXXXXXX

Step 2 : R
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****ZXX
X*XX*XX*XX
X*XX****OXX
X****XXXXXX
XXXXXXXXXXXX

Step 3 : D
XXXXXXXXXXXX
XXXXXXXXXXXX
X*****XX
X*XX*XX*XX
X*XX****ZXX
X****XXXXXX
XXXXXXXXXXXX

>> Waktu eksekusi: 0.129 ms
>> Banyak iterasi yang dilakukan: 5 iterasi

```

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3. | <p>Menguji urutan nomor yang dilewati<br/>File input:</p> <pre> 15 15 XXXXXXXXXXXXXXXXXX X7*6XXXXXXXXXXXXX X*X*XXXXXXXXXXXXX X*X*XXXXXXXXXXXXX X*X*XXXXXXXXXXXXX X*X*XXXXXXXXXXXXX X*X*XXXXXXXXXXXXX X*X*XXXXXX2*3XX X8***90XX*X*XX XXX*XZXXX*X*XX XXX5*****4XX XXXXX*XXX*XXX XXXXX0***1XXXX XXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXX 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 8 4 9 999 999 999 999 999 999 999 999 999 999 999 2 999 3 999 999 999 999 999 999 999 999 999 999 999 5 999 6 999 999 999 999 999 999 999 999 999 999 999 8 999 9 999 999 999 999 999 999 999 999 999 999 999 2 999 3 999 999 999 999 999 999 999 999 999 999 999 5 999 6 999 999 999 999 999 999 5 1 6 999 999 999 8 4 9 5 1 6 999 999 999 8 999 9 999 999 999 999 999 3 999 4 999 999 999 999 2 999 3 999 999 999 999 999 6 2 7 3 8 4 9 5 1 6 999 999 999 999 999 999 999 1 999 999 999 999 8 999 999 999 999 999 999 999 999 999 4 9 5 1 6 2 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 </pre> <p>Output:</p> <pre> &gt;&gt; Masukkan file input : test/testcase3.txt &gt;&gt; Algoritma apa yang anda pilih? (UCS/GBFS/A*) A* &gt;&gt; Heuristic apa yang anda pilih? (H1/H2/H3) H3  Solusi Yang Ditemukan : DRURDLULDR Cost dari Solusi : 248  &gt;&gt; Waktu eksekusi: 0.752 ms &gt;&gt; Banyak iterasi yang dilakukan: 26 iterasi </pre> |
| 4. | <p>Menguji Lava<br/>File input:</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

```
F testcase4.txt
4 7
XXXXXXXX
XZ*L*OX
X*****X
XXXXXXXX
999 999 999 999 999 999 999
999 1 1 999 1 5 999
999 1 1 1 1 1 999
999 999 999 999 999 999 999
```

Output:

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <pre> test/testcase4.txt &gt;&gt; Algoritma apa yang anda pilih? (UCS/GBFS/A*) GBFS &gt;&gt; Heuristic apa yang anda pilih? (H1/H2/H3) H1  Solusi Yang Ditemukan : DRU Cost dari Solusi : 10  Initial XXXXXXX XZ*L*OX X*****X XXXXXXX  Step 1 : D XXXXXXX X**L*OX XZ****X XXXXXXX  Step 2 : R XXXXXXX X**L*OX X****ZX XXXXXXX  Step 3 : U XXXXXXX X**L*ZX X*****X XXXXXXX  &gt;&gt; Waktu eksekusi: 0.091 ms &gt;&gt; Banyak iterasi yang dilakukan: 4 iterasi </pre> |
| 5. | <p>Menguji validasi input</p> <p>File input:</p>                                                                                                                                                                                                                                                                                                                                                                                                                      |

```

testcase5.txt
3 3
XXX
XZX
XXX
999 999 999
999 1 999
999 999 999

Output:
>> Masukan file input :
test/testcase5.txt
Error: Board harus memiliki tepat satu titik tujuan 0.

```

## 2. Analisis Hasil Pengujian

Berikut adalah tabel analisis hasil pengujian tiap testcase:

| TC          | Algoritma | Heuristic | Solusi                  | Cost | Waktu    | Iterasi |
|-------------|-----------|-----------|-------------------------|------|----------|---------|
| test case 1 | UCS       | -         | UDR<br>URU<br>LDR<br>UR | 146  | 0.262 ms | 18      |
| test case 2 | A*        | H1        | URD                     | 2    | 0.129 ms | 5       |
| test case 2 | GBFS      | H1        | R                       | 90   | 0.076 ms | 2       |
| test case 2 | UCS       | -         | DLU<br>RD               | 0    | 0.143 ms | 6       |
| test case 3 | A*        | H3        | DRU<br>RDL<br>ULD<br>R  | 248  | 1.111 ms | 26      |

|             |      |    |     |    |          |   |
|-------------|------|----|-----|----|----------|---|
| test case 4 | GBFS | H1 | DRU | 10 | 0.086 ms | 4 |
|-------------|------|----|-----|----|----------|---|

Pada testcase1, UCS menemukan solusi UDRURULDRUR dengan cost 146 dalam 18 iterasi. Testcase ini memuat angka urutan dan lava, sehingga algoritma tidak hanya perlu mencapai titik tujuan, tetapi juga harus melewati angka sesuai urutan dan menghindari jalur yang melewati lava.

Perbandingan paling jelas terlihat pada testcase2 karena input yang sama diuji menggunakan UCS, GBFS, dan A\*. GBFS menghasilkan solusi paling pendek, yaitu R, dengan waktu paling cepat 0.076 ms dan hanya 2 iterasi. Namun, cost solusi yang diperoleh adalah 90. Sebaliknya, UCS menghasilkan solusi DLURD dengan cost 0, tetapi membutuhkan 6 iterasi dan waktu 0.143 ms. A\* dengan heuristic H1 berada di tengah, yaitu menghasilkan solusi URD dengan cost 2, 5 iterasi, dan waktu 0.129 ms.

GBFS cenderung memilih jalur yang secara heuristic paling dekat dengan goal, tetapi tidak mempertimbangkan cost aktual yang sudah dikeluarkan. Akibatnya, GBFS dapat sangat cepat, tetapi solusi yang ditemukan tidak dijamin optimal terhadap cost.

UCS terbukti lebih baik ketika tujuan utama adalah memperoleh solusi dengan cost minimum. Pada testcase2, UCS menemukan cost paling kecil dibandingkan A\* dan GBFS. Hal ini sesuai dengan karakteristik UCS yang selalu memprioritaskan node dengan nilai cost aktual  $g(n)$  terkecil. Namun, karena UCS tidak menggunakan heuristic, algoritma ini dapat meninjau lebih banyak konfigurasi dibanding algoritma yang diarahkan oleh heuristic.

A\* menggunakan gabungan antara cost aktual dan estimasi heuristic. Pada testcase2, A\* menghasilkan jumlah iterasi dan waktu eksekusi yang sedikit lebih rendah daripada UCS, tetapi cost yang diperoleh lebih besar dari UCS. Hal ini menunjukkan bahwa heuristic H1 dapat membantu mempercepat pencarian, tetapi tidak selalu menjamin optimasi cost. Penyebabnya adalah H1 hanya memperkirakan kedekatan posisi secara geometris, sedangkan cost sebenarnya bergantung pada biaya tile yang dilewati selama sliding.

Pada testcase3, A\* dengan heuristic H3 menyelesaikan kasus yang lebih kompleks dengan solusi DRURDLULDR, cost 248, waktu 1.111 ms, dan 26

iterasi. Testcase ini memiliki papan lebih besar dan angka urutan 0 sampai 9, sehingga pencarian menjadi lebih sulit karena pencarian tidak hanya menyimpan posisi aktor, tetapi juga angka yang sudah dilewati. Oleh karena itu, testcase3 menghasilkan waktu dan jumlah iterasi terbesar di antara percobaan yang diberikan.

Pada testcase4, GBFS dengan H1 menemukan solusi DRU dengan cost 10, waktu 0.086 ms, dan 4 iterasi. Meskipun terdapat lava di dekat jalur awal, solusi yang ditemukan memutar melalui bawah sehingga tidak melewati tile lava. Hal ini menunjukkan bahwa validasi gerakan terhadap tile L sudah berjalan dengan baik.

Hasil percobaan memperlihatkan bahwa UCS lebih unggul untuk mencari solusi dengan cost minimum, GBFS lebih unggul dari sisi kecepatan dan jumlah iterasi tetapi tidak optimal, sedangkan A\* berada di antara keduanya. Namun, kualitas A\* sangat bergantung pada heuristic yang digunakan. Untuk kasus dengan cost tile yang bervariasi, heuristic berbasis jarak seperti Manhattan distance belum tentu cukup akurat karena tidak memperhitungkan bobot tile.

## Lampiran

Link Github: [https://github.com/wzlyy/Tucil3\\_13524054](https://github.com/wzlyy/Tucil3_13524054)

Pernyataan penggunaan AI:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Wildan Abdurrahman Ghazali

Tabel pengerjaan tugas:

| No | Poin                                                                                                    | Ya | Tidak |
|----|---------------------------------------------------------------------------------------------------------|----|-------|
| 1  | Program berhasil di kompilasi tanpa kesalahan                                                           | ✓  |       |
| 2  | Program berhasil dijalankan                                                                             | ✓  |       |
| 3  | Solusi yang diberikan program benar dan mematuhi aturan permainan                                       | ✓  |       |
| 4  | Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt                      | ✓  |       |
| 5  | Program memiliki Graphical User Interface (GUI)                                                         |    | ✓     |
| 6  | Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y) |    | ✓     |
| 7  | Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)                  | ✓  |       |